

FaaSRep: Enabling Strict Serializability with Efficiency for Stateful Serverless Workflows via Transaction Repair

Anonymous Author(s)

Abstract

Serverless computing, represented by Function-as-a-Service (FaaS), is increasingly adopted for deploying complex applications via stateful workflows. However, the lack of strong consistency (i.e., strict serializability) transaction support hinders its broader applicability. Existing solutions are inadequate, either offering only weak consistency or suffering severe performance degradation from concurrency control models that blindly retry executions after conflicts.

We therefore present FaaSRep, a high-performance serverless framework that provides strict serializability-level transaction support through *transaction repair*. Instead of blindly retrying failed transactions, FaaSRep constructs a data dependency graph after initial execution, and by which it orchestrates an organized retry to 'repair' the conflicts. FaaSRep introduces three key techniques: (1) a hybrid repair model that correctly handles proactive application aborts; (2) delayed dependency construction at validation time to avoid communication overhead; and (3) fine-grained function-level repair equipped with a peer-to-peer communication fast-path. When comparing to state-of-the-art systems under real-world workloads, our evaluation shows that FaaSRep improves average throughput to 2.03-3.69 $\times$ for Concord and 4.55-9.01 $\times$ for Beldi.

1 Introduction

Serverless computing, also known as *Function-as-a-Service* (FaaS), has gained widespread adoption in the cloud-native era thanks to its advantages such as ease of use and pay-as-you-go billing [15, 38, 57]. To support complex applications like data analytics pipelines [19, 20, 25] and machine learning inference chains [18, 24], modern serverless platforms have evolved beyond simple isolated functions to support *serverless workflows*. These workflows are typically structured as Directed Acyclic Graphs (DAGs), where nodes represent functions and edges denote invocation dependencies [44, 48, 53]. In response to this trend, major cloud providers have introduced orchestration services such as AWS Step Functions [2] and Azure Durable Functions [4], making serverless workflows a foundational building block for large-scale cloud applications.

Despite the great potential, today's FaaS platforms struggle to support those applications requiring external states with strong data consistency guarantees [72, 76], restricting the applicability. For instance, online shopping business [11, 13] must update inventory counts serially when serving concurrent requests, and avoid showing any changes in inventory counts caused by unsubmitted purchases, to prevent overselling. Such workflows require running in a *transaction* abstraction with *strict serializability* to execute correctly. Transaction ensures that state operations either all succeed or all fail [32, 43, 70], while strict serializability guarantees concurrency safety respecting real-time constraints [23]. The absence of these features shifts the complex burden of enforcing transactional guarantees to developers.

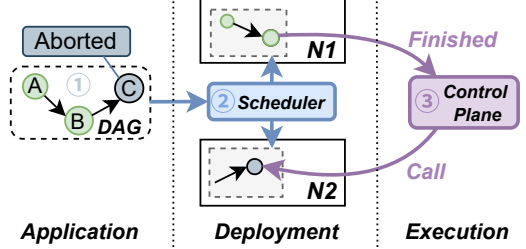


Figure 1: The architecture of the serverless platform and the transactional workflow.

While some prior works introduced transactional workflow, they achieve strong consistency with high overhead [61, 63, 72]. The inefficiency mainly stems from their concurrency control models, such as two-phase locking (2PL) [36] and optimistic concurrency control (OCC) [42], and can be categorized as **contention-based** models. In these models, each transaction freely competes for data. When a conflict is detected through a failed lock acquisition by lower-priority transactions (in 2PL) or validation after execution (in OCC), the only recourse is to abort and blindly retry the transaction. This reactive strategy leads to severe performance degradation, with our experiments revealing that retry overhead leads to more than 4 $\times$ of latency increment. (§3.2)

Instead of blindly retrying, a promising alternative is to orchestrate transactions based on a holistic view of data dependencies, executing conflicting operations in a strict serial and well-organized order. However, predicting transactions' access sets to construct dependencies in advance is difficult. A viable strategy is the repair-based paradigm applied in the distributed database domain [29, 31]. This approach first performs a *trial execution* to capture conflicts between concurrent transactions, integrating these dependencies into a global graph. Although serializability violations may occur during this phase, the graph enables an *organized retry*: transactions are strictly re-executed in the graph's topological order. By ensuring each transaction reads valid, up-to-date data from its predecessors, this deterministic process effectively "repairs" the inconsistencies introduced during the uncoordinated trial execution.

However, it is nontrivial to achieve transaction repair in a serverless environment, whose typical architecture can be summarized as three layers shown in Figure 1. First, the *application layer* (①) defines function logic and the DAG expression of workflow [15, 44], but the function code uploaded by users often contains logic of proactive abort [72] (e.g., insufficient balance during transfer in the banking system). When a predecessor transaction aborts, it invalidates the serialization order established by its dependent successors, triggering cascading aborts throughout the graph. The cost of such cascading aborts is unpredictable, since it scales with the number of dependent transactions and occurs whenever a transaction aborts.

Second, the deployment layer (2) decides placement of functions across physical nodes through the scheduler (e.g., functions A and B on Node1 (N1), C on N2), and can cause the execution context of a serverless workflow spanning multiple nodes [48, 53]. At this time, constructing a global dependency graph at runtime as in prior work [31] incurs substantial remote access overhead, since it requires querying all nodes or maintaining a centralized log in this decentralized situation, both of which undermine performance.

Third, the *execution layer* (3) manages workflow execution via the control plane [45, 48]. As depicted in Figure 1, upon completion of function B, the control plane is notified to schedule and trigger function C on an available container. Such indirect communication introduces non-negligible scheduling overhead [44, 48], which is further amplified by repair, as each re-execution step still requires control plane intervention. Moreover, when a transaction is retried, all its constituent functions are re-executed regardless of whether they were actually involved in any conflict. This coarse-grained approach leads to unnecessary re-execution of unaffected functions, wasting compute resources and reducing overall system efficiency.

Therefore, we present **FaaSRep**, a high-performance transactional system for serverless workflows based on the idea of repair. During initial execution, FaaSRep acts like OCC-based systems [16], accelerating reads through cache and submits batched requests for validation. Next, it orchestrates an organized retry to repair conflicts using the dependency graph. To handle proactive aborts, FaaSRep introduces a *hybrid repair model* that starts with an *optimistic mode* and gracefully falls back to a *pessimistic mode* only when a predecessor proactively aborts, balancing correctness and performance¹. To avoid runtime overhead, FaaSRep *delays construction of dependency graph* at its validation phase, to naturally gets a global view of data accesses². Furthermore, FaaSRep performs *function-level repair* for fine granularity and introduces a peer-to-peer *fast-path* communication channel between containers to efficiently bypass the control plane³ during repair. By leveraging container placement information from the initial run, FaaSRep's communication channel avoids the handshake overhead seen in similar approaches [48]. The contributions of this paper are as follows:

- **The paradigm comparison and analysis of different concurrency models.** The analysis identifies key factors that hinder performance in existing transactional serverless systems, and shows the potential of repair in FaaSRep design.
- **The protocol design of transaction repair in serverless environment.** We design a set of key techniques to efficiently introduce hybrid transaction repair into the unique serverless environment and ensure the strict serializability of the repair protocol.
- **A complete implementation of FaaSRep runtime system based on the transaction repair.** We further implement function-level repair and fast-path communication for serverless transactional workflow system that achieves high performance and low overhead.

Our evaluation on a 8-node cluster shows that FaaSRep significantly outperforms state-of-the-art baselines. Evaluated with real-world workloads, FaaSRep achieves up to 3.69× higher throughput than Concord, and 9.01× higher than Beldi, with the same consistency guarantee of strict serializability.

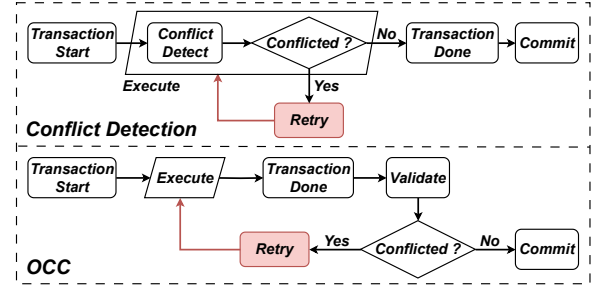


Figure 2: Explanation of existing solutions.

2 Related Work

Transactional support in serverless computing with strong consistency. Beldi [72] implements a two-phase locking (2PL) protocol to ensure strict serializability, maintaining locks in a global storage service. For efficiency, Concord [63] utilizes a directory-based cache coherence protocol, while Boki [37] and Halfmoon [56] record lock entries in a shared log abstraction. However, as Boki and Halfmoon periodically update log indices by subscribing to the storage layer, they only provide *snapshot isolation* [22], which prevents concurrency problems like phantom reads but is not serializable. Another work, RTSFaaS [76], exploits a lease-based concurrency control protocol, but requiring knowledge of the entire access set in advance to build its task-precedence graph.

Serverless transactions with weak consistency. A series of works turns to weak consistency guarantees to prevent the overhead caused by maintaining strong consistency [52, 61, 62, 69, 73]. Among them, AFT [61] deploys a fault-tolerance shim between the computing and storage layers, offering *Read Atomicity* [30] guarantees. CloudBurst [62] and FaaSSTCC [52] offer *Transactional Causal Consistency* guarantees by checking whether the causal dependency is violated during runtime. All these approaches cannot prevent concurrent write conflicts, thereby violating serializability.

Transaction support in the traditional database domain. Some works focus on transactions on a single database node [26, 36, 47, 67]. To improve the 2PL parallelism, Bamboo [36] allows transactions to release locks early. For distributed databases [31, 64–66], Calvin [65] avoids 2PC overhead by pre-ordering transactions before execution, while limiting concurrency. And Hackwrench [31] also uses a transaction repair-based approach via dependencies tracked during execution, but it assumes that transactions will not abort proactively. The transactions in these works are pure data operations, incompatible with serverless functions.

3 Background and Motivation

3.1 Transactional Serverless Workflow

Applications in serverless computing are often structured as workflows of functions, and represented as Directed Acyclic Graphs (DAGs). To achieve load balancing [33, 39], these functions may be scheduled to execute on different nodes, forming a cross-node execution context [44, 48]. Generally, they are unaware of each other's location, as function instances are created on demand by the node controller [15, 38, 46]. During runtime, the serverless workflow relies on the control plane to route invocation requests [44].

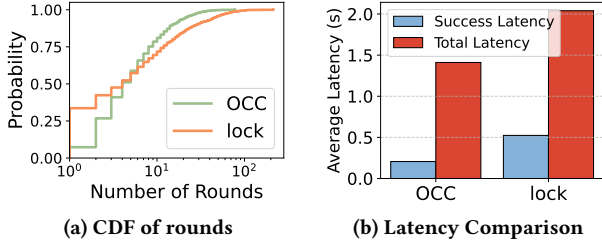


Figure 3: Execution results on remote locking and OCC.

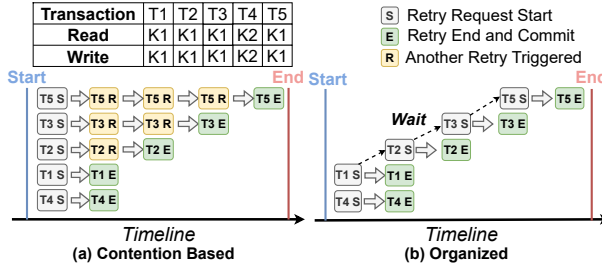


Figure 4: Transactions retry with different schemes.

While stateless workflows such as video processing [14] and content moderation [12] are common in serverless, a wide range of applications such as financial systems [8] rely heavily on external state and require transaction abstraction with strict serializability to function correctly [1, 34]. This guarantee, analogous to ACID transactions in traditional databases [32], provides the strongest data consistency: it ensures that the global effect of multiple concurrently executing requests is equivalent to a sequential execution. Furthermore, it enforces the real-time order constraint, meaning that if one transaction commits before another begins, it must appear earlier in the serial order. Supporting these transactional guarantees is essential for expanding the applicability of serverless architectures to more complex and critical domains.

3.2 Implications of Existing Solutions

Current serverless platforms typically offload state to external storage (e.g., AWS DynamoDB [7]) but lack native transactional support. For prior works that address this, their execution flows are shown in Figure 2. Efforts such as Beldi [72], Boki [37], and Concord [63] focus on detecting and preventing conflicts during execution. For similar attempts in the database domain, a more common approach is *Optimistic Concurrency Control* (OCC) [16, 42, 67]. It allows transactions to read from cache during execution optimistically and performs a validation phase to check conflicts. Furthermore, it batches multiple transactions in validation to reduce network overhead [66].

Essentially, these works run transactions in a contention-based model and repeatedly abort and retry those that fail in contention (red arrow in Figure 2). This passive approach causes certain transactions to get stuck in failed attempts, degrading system performance. This limitation extends even to protocols that rely on waiting, such as Two-Phase Locking (2PL) used in Beldi and Boki, because their

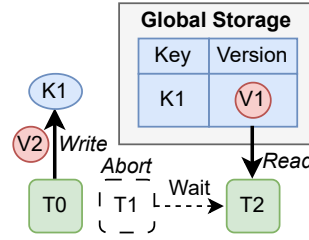


Figure 5: Transaction T_1 aborts, and T_2 misses T_0 's update.

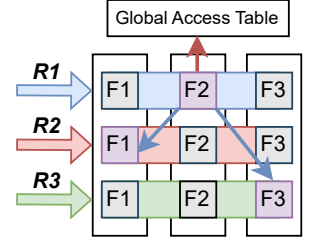


Figure 6: Requests R_1 , R_2 and R_3 form a cross-node access pattern.

deadlock prevention schemes, like Wait-Die [58], still force lower-priority transactions to abort upon lock contention.

To explain, we tested a travel reservation workload [1] on remote locking used by Beldi and OCC, following the setting of Section 8.6. According to Figure 3a, approximately 40% of the requests required more than ten attempts to complete in both methods. Figure 3b further reveals that the average latency to finish execution only accounts for 14.6% and 25.7% in the total average end-to-end latency, respectively. In both methods, the majority of time is spent in continuous cycles of abort and retry due to contention.

3.3 Opportunity of Organized Retry by Repair

To avoid the costly contention-based model (red in Figure 2), a promising approach is to perform *organized retry*. *Transaction repair* serves as an available paradigm for this. It first constructs a dependency graph from the collected read/write sets of the transactions. During the retry phase, conflicting transactions can start execution in an organized topological order simply by waiting for their upstreams to complete, thereby deterministically resolving conflicts and achieving serializability.

Figure 4 illustrates its benefits in a hotspot update scenario where transactions T_1 – T_5 contend for shared resources. With their access sets known, T_2 , T_3 , and T_5 are effectively serialized, while T_4 , having no conflicts, can run concurrently.

In this manner, non-conflicting transactions can be retried in parallel, while conflicting ones are serialized following the dependency graph's topological sort, and only need one retry to complete successfully. In other words, we could “**Repair**” the damage caused by contention rather than discarding the entire transaction, thereby eliminating fruitless retries, releasing system resources to serve new incoming requests, thus improving throughput.

3.4 Challenges for Leveraging Repair

However, it is nontrivial to design a repair-based scheme for serverless environments, due to the incompatibilities of repair with the characteristics and application patterns of serverless.

3.4.1 Application layer: Presence of Proactive Aborts. The serialized guarantee of repair may be violated if a transaction triggers a proactive abort. Figure 5 illustrates such a scenario. While T_1 is waiting for T_2 as organized by the dependency graph, there is another transaction T_0 that writes K_1 but hasn't committed yet. When T_1 aborts, to keep serializability, T_2 faces two unenviable

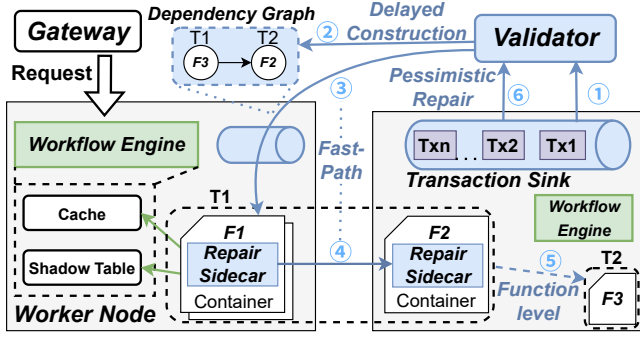


Figure 7: Architecture overview of FaaSRep.

choices: either locate $T0$ and read its value, or abort itself and rebuild the serialization order. The former requires costly distributed queries and validation across nodes, while the latter can trigger potentially unbounded cascading aborts [36]. Both options impose hard-to-bound and unpredictable overheads.

Unfortunately, such proactive aborts are not rare corner cases in serverless workflows, as applications often include explicit conditions that intentionally terminate execution. Typical scenarios include cancelling a purchase due to insufficient account balance or aborting a login after failed authentication checks [5, 34, 72].

Challenge 1: Transaction repair cannot handle proactive aborts, a feature presenting in many real-world applications. A more robust mechanism is needed to enhance generality.

3.4.2 Deploying layer: Cross-Node Characteristic. Commercial platforms like AWS Lambda [2] dynamically schedule function invocations across a fleet of isolated compute nodes. Since complex workflows frequently involve high parallelism (spanning up to 37 concurrent functions in Azure trace [54]), their execution contexts are often distributed across multiple physical nodes. This inherent cross-node nature makes it harder to capture the global data dependencies used for repair because concurrent accesses to the same data item are also physically dispersed.

Figure 6 illustrates this scenario with three concurrent transactional workflow requests ($R1$, $R2$, and $R3$) executing functions ($F2$, $F1$, and $F3$, respectively) on different nodes. To construct dependencies for $R1$ in this distributed environment, we are forced to adopt expensive coordination mechanisms: either querying all competitor nodes (Blue arrows) or accessing a centralized service for global access logs (Red arrows). These methods face similar remote communication overhead as the remote lock design in Beldi [72], accounting for more than 25% of the total latency (Section 8.7).

Challenge 2: The distributed deployment of serverless workflows makes it difficult to construct data dependencies at runtime, and naive construction methods lead to performance degradation.

3.4.3 Execution Layer: Overhead and Granularity. Serverless control plane introduces non-negligible indirect communication overhead, also known as scheduling overhead. According to previous studies, this overhead can reach 181.3ms for real-world applications [44]. When we perform transaction repair, this overhead is further amplified, as the repair execution of a workflow is also orchestrated and forwarded by the control plane.

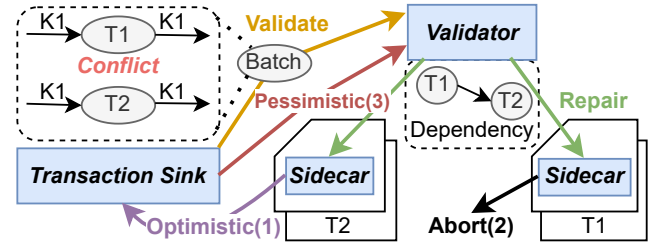


Figure 8: The repair workflow of FaaSRep.

Furthermore, as each function in the transaction has distinct upstream writers, conflicts can arise only for a subset of all functions. Therefore, simply retrying the entire transaction like prior works [63, 72] to perform repair results in redundant computations, as unaffected functions that encounter no conflicts and produce correct results are still re-executed, hindering system efficiency.

Challenge 3: Transaction repair exacerbates scheduling overhead, slowing down overall system performance. Additionally, a fine-grained repair solution tailored for the serverless context is required.

4 FaaSRep Overview

4.1 Architecture

Figure 7 presents FaaSRep, a system enabling an efficient repair scheme in serverless environment. In its architecture, each worker node runs a *workflow engine* to orchestrate workflow execution, managing a shared cache and an application-specific shadow table [72]. Within each container, a *repair sidecar* provides FaaSRep's runtime APIs, including `read(key)`, `write(key, value)`, and `abort()`, allowing users to build stateful apps. Besides, the *transaction sink* and *validator* participate in the repair and commit process.

During execution, FaaSRep employs an Optimistic Concurrency Control (OCC) strategy, accelerating reads through local cache and buffering writes to the shadow table for isolation. Throughout the workflow, read/write sets are propagated along the DAG and collected by the *transaction sink*. Periodically, the transaction sink batches these sets and forwards them to the external *validator*, which performs a validation phase to detect conflicts [42].

FaaSRep's repair process serves as OCC's extension, ensuring serialization during retries to fix the inconsistencies occurring in the first run. It begins when the validator receives a batch from the transaction sink① and constructs a dependency graph②. Guided by which, repair sidecars orchestrate re-execution via a peer-to-peer *fast-path*③④. According to the dependency graph, each function independently waits for upstream dependencies to finish before re-execution⑤, thus executing in a conflict-free order. After all transactions in a batch have been repaired, their buffered write sets in shadow tables are committed to storage. In rare cases requiring *pessimistic repair*, the transaction sink resubmits the affected transactions for a new round⑥.

Figure 8 shows a more specific example of FaaSRep's repair process: Transaction $T2$ performs an optimistic repair (1) but is invalidated by $T1$'s abort (2), thus forced to conduct another pessimistic repair round (3).

provides a *pessimistic repair mode*. It defers dependency construction until a transaction becomes *deterministic*, i.e., all relevant predecessors are repaired or aborted. Under this case, the dependencies of a transaction will not be invalidated anymore, keeping it away from the risk brought by proactive aborts.

However, waiting for all upstreams is inefficient. To maximize parallelism, FaaSRep determines the earliest deterministic point for each transaction, shown in Figure 10. For each transaction T and its batch B , FaaSRep records the nearest batch to B that contains T 's dependencies to be T 's *pessimistic batch dependency* (PBD). Within batch B , FaaSRep marks T 's nearest upstream transaction as its *pessimistic transaction dependency* (PTD). T becomes *pessimistic-ready* and safe for repair when all batches before its PBD are committed and all transactions before its PTD are resolved. The correctness is explained in Section 5.5.

The validator continually updates these dependencies as validation proceeds. To determine who is the "nearest", FaaSRep leverages timestamps for other batches and validation order for transactions within the same batch. After validation, it records the dependencies in the transaction sink, which monitors when a transaction becomes pessimistic-ready and notifies the validator to trigger repair. Aborted transactions have invalidated write sets and are excluded from dependencies.

When constructing dependencies for a pessimistic-ready transaction, the check for writers can be confined to its own batch, considering only the transactions before its PTD, as all possible upstream batches have been committed. This manner avoids unbounded backtracking when a predecessor aborts.

5.4 Fallback from Optimistic to Pessimistic

To balance system throughput and correctness, FaaSRep initially processes all transactions using *optimistic-repair*. And if a transaction triggers a proactive abort during repair, its successors are immediately downgraded to *pessimistic-repair* mode.

The transaction sink manages this transition. It explicitly maintains a *repair mode* for each transaction alongside its completion status. When an abort occurs during execution, the control plane reports this event to the sink. Upon receiving the abort signal, the sink cascades the mode switch to all dependent transactions, ensuring they are rescheduled under the pessimistic protocol.

This hybrid approach introduces a potential consistency risk: a transaction might commit its optimistic result just as a predecessor aborts. To address this, FaaSRep only allows an optimistic transaction to commit if it satisfies the *pessimistic-ready* condition, which ensures that it is safe from any preceding aborts. If a predecessor aborts during this window, the transaction must discard its optimistic result and wait to be retried under pessimistic repair.

While this fallback mechanism may result in wasted optimistic executions, the overhead is strictly bounded. Due to the determinism of the pessimistic mode, a request will undergo at most three runs: the initial execution, an optimistic repair, and a pessimistic repair. This holds true even in the presence of frequent aborts.

5.5 Correctness Analysis

Theoretically, FaaSRep's repair protocol ensures strict serializability. The proof sketch is based on the following notations.

Serializable: For all transactions executed, their reads and writes are equivalent to a certain serial execution sequence S .

Strictly Serializable: Besides being serializable, for any two transactions T_1 and T_2 that T_1 commits before T_2 starts, T_1 must be before T_2 in the serial execution sequence S .

Dependency: If in the equivalent serial sequence, transaction T_2 reads a data item written by transaction T_1 , then we say that T_2 depends on T_1 , denoted as $T_1 \rightarrow T_2$.

OBSERVATION 1. *If all transactions are validated in a single batch, then optimistic repair mode is serializable.*

BASIS: We denote the equivalent serial sequence S to be the validation order of transactions, noted as $[T_1, T_2, \dots, T_n]$. If in S , $T_j \rightarrow T_i$ ($j < i$), then T_i waits for T_j to complete and read the data written by T_j during repair. As FaaSRep commits using the merged write set, the global state reflects the modifications made by S , but directly turns to the final state. As reads and writes all match those in S , the optimistic repair mode is serializable. $\square$

OBSERVATION 2. *If transactions are processed in multiple batches, Serializability is still maintained.*

BASIS: For multiple batches, we can get a new serial order S' : $[T_1^1, T_2^1, \dots, T_n^1], \dots, [T_1^2, T_2^2, \dots, T_m^2]$. As FaaSRep commits batches serially, the only issue is whether the reads in repair match S' .

For a transaction T_n^m , suppose in S' it reads key k written by T_b^a ($a < m$) in a different batch, when validating T_n^m , if T_b^a has already been committed, all writers before T_b^a have been committed, making writers list for k empty. At this time, T_n^m reads the latest value from the global storage, which is from T_b^a .

In the other case, if T_b^a has not been committed, the dependency graph of T_n^m contains an edge from T_b^a . During retry, it fetches value from T_b^a 's shadow table, same as S' . Thus, in both cases, T_n^m reads the same value as in the serial order S' . $\square$

THEOREM 1. *Optimistic repair mode is strictly serializable.*

BASIS: The optimistic repair is serializable with Observations 1 and 2 combined. Now, for transactions T_1 and T_2 such that T_1 commits before T_2 starts, they must be in different batches, as FaaSRep commits a batch at once. Then, FaaSRep's sequential validation ensures that T_2 's batch observes a snapshot that includes T_1 's commit, showing T_2 must be after T_1 in the equivalent serial order. $\square$

THEOREM 2. *Pessimistic repair mode is strictly serializable.*

BASIS: If there are no aborts, the pessimistic repair mode keeps strict serializability with the optimistic mode. As they follow the same validation procedure. Then, for transactions T_a and T_b , suppose $T_a \in B_a$, $T_b \in B_b : [T_1, T_2, \dots, T_x, \dots, T_b]$, $T_a \rightarrow T_b$, and T_a aborts during its repair. Let T_b 's PTD be T_x and PBT be B_x respectively.

If $B_a \neq B_b$, then B_a is before B_x . When T_b becomes pessimistic-ready, batches from B_a to B_x have been committed, matching Observation 2. So T_b reads the consistent data with the serial order.

If $B_a = B_b$, then T_a is before T_x . When T_b becomes pessimistic-ready, transactions from T_1 to T_x have been repaired or aborted, including T_a . As T_a has aborted, T_b will skip T_a when constructing dependencies. Therefore, strict serializability is preserved. $\square$

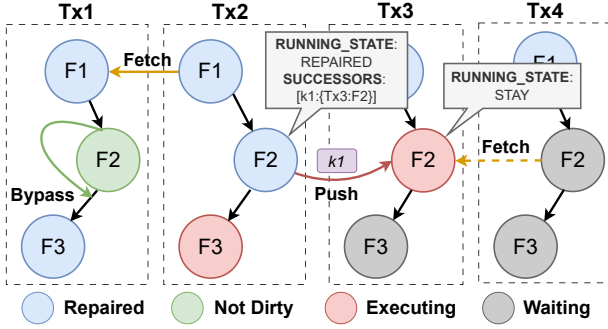


Figure 11: Function-level repair procedure.

6 Efficient and Fine-Grained Repair

In this section, we present our designs for handling repair at function granularity, and building a fast-path to bypass control plane.

6.1 Repair at Function granularity

To align with the serverless execution model [44], FaaSRep treats each function within a transaction as the basic unit of repair to prevent redundant computations. The fine-grained repair process is illustrated in Figure 11. Before the execution of each function, the workflow engine checks the dirty flag in dependency graph, to skip its execution or prepare its dependencies. For the latter case, the workflow engine fetches all data items recorded in DependencyInfo from upstream functions' shadow tables. When a function attempts to read the corresponding upstream data, its repair sidecar transparently intercepts the call and provides the correct data version that was pre-fetched, remaining invisible to user logic.

It's worth noting that a function's upstream may have not been repaired when the function starts executing. To address this, FaaSRep explicitly manages each function's execution state, including its RUNNING_STATE and SUCCESSORS.

RUNNING_STATE indicates whether the function has completed repair, which can be STAY or REPAIRED. SUCCESSORS indicates downstream waiters. Each function first queries the RUNNING_STATE of its upstream during repair. If the state is STAY, the function leaves information to the upstream function's SUCCESSORS list and waits. When the upstream function completes its repair, it changes its state to REPAIRED, checks its current successors list, and proactively pushes the required data to the nodes of the downstream waiters. Skipped functions also perform this to wake up their successors.

6.2 Fast-Path for Repair

In serverless workflow's standard execution model, each function call traverses a multi-hop path: from the caller to a workflow engine, potentially between engines, and finally to the target container [44]. This "slow-path" is ill-suited for the rapid, fine-grained communication required during transaction repair, as it would amplify latency and place immense pressure on workflow managers [48].

To address this, FaaSRep introduces a **fast-path** communication mechanism specifically for the repair phase, as illustrated in Figure 12. It establishes an end-to-end communication channel among the function containers involved in a transaction batch. After initial

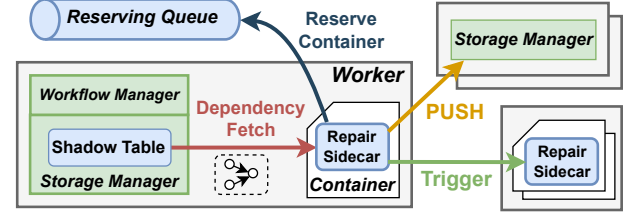


Figure 12: Fast-path repair illustration.

execution, the workflow engine places the used function containers into a dedicated reserving pool, preventing them from being reclaimed [49]. Crucially, it also collects the network endpoints (IP and port) of each container's repair sidecar. This topology information is then merged with the dependency graph during validation.

During repair, instead of routing calls through the control plane, the repair sidecar leverages the placement information to invoke functions directly, reducing the network path to a single hop between containers. The dependency information itself is pre-staged on the worker nodes, allowing the sidecar to fetch locally before repair, further minimizing network traffic on the critical path.

For large-scale deployments where cross-node fetching for dependencies (Section 6.1) could become a bottleneck, the fast-path design can be integrated with low-latency networking technologies like RDMA [35, 41]. And all fetching and pushing operations can be optimized using one-sided RDMA primitives [28, 40, 51].

Specifically, the receiver could maintain a circular buffer [74, 76] for incoming data items, and register a dedicated thread to poll the buffer and wake up waiters. For senders, they directly write data into the receiver's buffer. This minimizes CPU involvement and network stack overhead, drastically reducing data transfer latency.

7 Implementation

FaaSRep is implemented on top of FaaSFlow [45], whose distributed workflow manager reduces cross-node traffic. We extend FaaSFlow's workflow state to include each function's read/write sets and the network endpoints of its containers.

Our implementation distinguishes between two primary communication paths. The invocation path used in the initial run relies on the control plane triggering functions via standard HTTP REST APIs. In contrast, the "fast-path" for the repair phase is implemented as a peer-to-peer network using gRPC [68]. To ensure security, the fast-path gRPC strictly limits communication to containers within the same workflow. Each function runs in an isolated Docker [6] container, with a repair sidecar packed in the image. This sidecar acts as both a gRPC server and client.

Each worker node hosts a Redis instance, partitioned to serve as both a local cache and a shadow table. Functions access this local Redis [9] instance and global database through their sidecar. The checking and updating of RUNNING_STATE and SUCCESSORS are implemented using Redis atomic operations to ensure correctness. The validator is implemented as a scalable event [7] WSGI server combined with a process pool to handle concurrent requests.

7.1 High-scalable Validator

FaaSRep tries to decouple the validator, to prevent it from becoming a scalability bottleneck in high-concurrency environments. Based on real-world data [57] and insights from previous work [63], we first observed that serverless applications exhibit a high degree of data isolation, with 99.7% of data not being shared across multiple applications. Consequently, FaaSRep constructs validators at the application granularity, providing each application with its own dedicated validator and transaction sink instance. This model can seamlessly scale up or down as the number of applications changes.

However, even a dedicated validator is still limited by its single-instance nature. Therefore, FaaSRep further decouples the validator to maximize scalability and parallelism. A FaaSRep validator is configured with a single-instance *serializer* and multiple, scalable *processors*. Besides, a proxy is responsible for distributing incoming batch validation requests to different processors.

Serializer maintains the global table mentioned in Section 5.1, sequentially checks versions and finds dependencies. Multiple processors concurrently receive batch validation requests and prepare preliminary data, interacting with the serializer via a work queue. After receiving results from the serializer, they proceed with subsequent operations such as constructing repair metadata and sending repair requests. After repair, the serializer follows the order of writers to commit batches.

7.2 Failure Recovery

For the storage layer, FaaSRep relies on the underlying database's own fault tolerance mechanisms [59]. Here, we focus on failures of compute nodes and the FaaSRep validator.

At runtime, FaaSRep continuously monitors the status of each node. If a crash is detected, The validator directly discards all uncommitted transactions, including those currently being retried or awaiting validation, and retry them after recovery. Transactions that have already been committed are unaffected, as their modifications have been fully recorded.

Some batches may be interrupted during the commit phase, resulting in partially committed data. To address this, FaaSRep logs every batch's modifications in the database before committing, denoted as *delta set*. Once the commit is fully completed, a completion log is recorded. During recovery, any batch without this log recorded must roll back its changes.

When a validator crashes, all validation and repair operations of the corresponding application are paused. During recovery, the validator contacts the gateway to identify any uncommitted transactions and retrieves their corresponding delta set logs. After rolling back modifications, The validator reconstructs its global table.

7.3 Limitation: Determinism of Transaction

FaaSRep's repair mechanism relies on the assumption of **determinism**: given the same inputs, a function must generate an identical execution flow and read/write set during repair. However, real-world workflows often incorporate non-deterministic elements like randomness, ML inference, or external API responses, which may alter execution paths in repair, thus invalidating the correctness.

This imposes a strict transaction boundary. FaaSRep guarantees serializability only for states accessed through its data APIs

and is not designed to wrap serverless applications that mix with uncontrolled side-effects. Instead, it can serve as a specialized consistency layer for critical, inherently deterministic transactional cores in a larger workflow, such as inventory management or balance transfers. While prior studies indicate that such static patterns are prevalent [37, 72, 76], developers should explicitly decouple non-deterministic operations from this core, leaving them to alternative consistency layers, forming a multi-consistency architecture [60].

8 Evaluation

In this section, we evaluate the performance and effectiveness of FaaSRep in detail. Our evaluation includes its performance under various configurations and workloads, the effectiveness of its mechanism, and the overhead it introduces.

8.1 Experimental Setup

We conduct experiments on a cluster of 8 nodes, with one storage node and seven worker nodes. Each node is equipped with an Intel(R) Xeon(R) Platinum 8562Y+ CPU, having 32 cores and 256GB memory. Functions runs in Docker container instances, limited to use at most 1 CPU core and 256MB of memory.

The storage node is deployed with a gateway, storage backend, FaaSRep validator, and clients generating requests. We use ScyllaDB [10] as the global storage backend, storing data items, metadata and logs. The DB instance is configured to use at most 20 CPU cores with the strong consistency read option enabled.

In most sections, we configured different numbers of clients to send requests in a closed-loop manner. The number of clients is adjusted to control the overall request concurrency in the system. Besides, in Section 8.6, we include an experiment that employs an open-loop approach to simulate the Azure trace [75].

Baselines: We compare FaaSRep with two strict serializable systems: Beldi [72] and Concord [63]. For Beldi, We implemented a Wait-Die scheme for deadlock prevention [72]. For Concord, we use dedicated HTTP servers as cache managers. To improve performance, both systems are configured to allow concurrent reads, similar to read-write locks. Additionally, we implement an OCC system based on the same principles discussed in Section 3.2, serving as a downgraded version of FaaSRep. To ensure a fair comparison, we implement all systems on the same FaaSFlow [45] codebase.

We use two types of workloads in our evaluation: a micro-benchmark implemented by ourselves and several real-world applications adopted from prior works [5, 37, 72, 76].

Micro-benchmark: The micro-benchmark contains two sets of transactions: sequence-chain-like transactions and transactions with high fan-out, with varying lengths of 2, 4, 8, 16 (**noted as** c_2, c_4, c_8, c_{16}) and fan-out degrees of 2, 4, 6, 8 (**noted as** w_2, w_4, w_6, w_8) respectively. They access 4KB texts from a dataset of 10000 keys, following Zipf distribution [21].

Travel reservation (*Travel*): It involves functions of booking a flight, renting a car, and confirming the payment. Abort occurs if any of the remaining resources are insufficient.

Banking system (*Bank*): It simulates the transfer process between two bank accounts. A function first checks the balance of the source account, then deducts the amount. The following function adds it to the destination account.

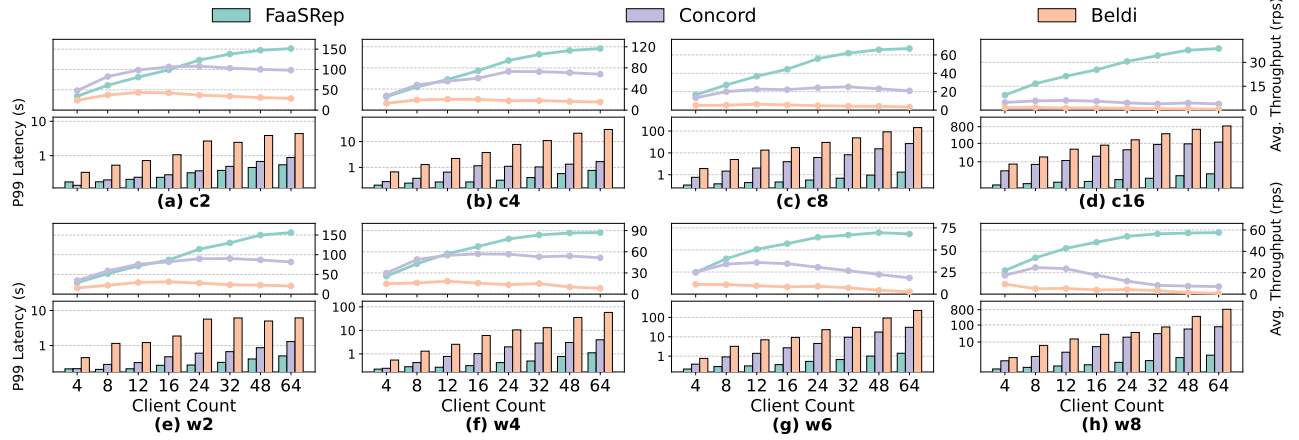


Figure 13: P99 latency and average throughput for transactions with different lengths and fan-out degrees.

Social media (Social): Users log in, then comment on several posts and publish a new post under their timelines. The transaction ensures that all comments are published atomically.

8.2 Performance of FaaSRep

This experiment shows FaaSRep’s effectiveness across various application styles using our micro-benchmark. We set the Zipf factor to 0.9, following mainstream configurations [27, 55, 65], and the read/write ratio in transactions is 2:1 to simulate a hybrid pattern.

Figure 13 shows the result. FaaSRep consistently outperforms both baselines in the majority of tested scenarios. In the moderate concurrency (e.g., 32 clients), FaaSRep surpasses Concord (Beldi) by $1.33\text{--}8.81\times$ ($4.06\text{--}41.35\times$) in throughput, while reducing the p99 latency by more than 23.58% (84.72%) compared to Concord (Beldi). This demonstrates that FaaSRep’s repair-based approach effectively mitigates the performance degradation. Concord generally outperforms Beldi, thanks to its efficient cache coherence protocol.

As the client count or scale of transactions increases, the performance gap widens because contention becomes more severe and the cost of aborts rises. In extreme scenarios (e.g., c16 with 64 clients), Beldi and Concord are nearly unusable, with the throughput lower than 4 rps. Across all settings, Beldi consistently performs worst, largely due to its lack of data locality, even when database pressure is alleviated under low-load cases (such as c2 with 4 clients).

The two baselines perform worse for high fan-out transactions, which involve more concurrent data accesses and higher contention for shared resources. At the same time, the variation trends of performance remain stable across all style applications in FaaSRep, reaching the peak throughput at 64 clients from c2 to w16.

However, FaaSRep’s advantages come with a trade-off in low-contention scenarios. For instance, in the c2 workflow with 4 clients, its throughput is $0.71\times$ that of Concord because the repair overhead is not amortized by infrequent conflicts. Nevertheless, such low-parallelism environments are rare, as real-world serverless workloads typically exhibit higher concurrency [48, 50, 57].

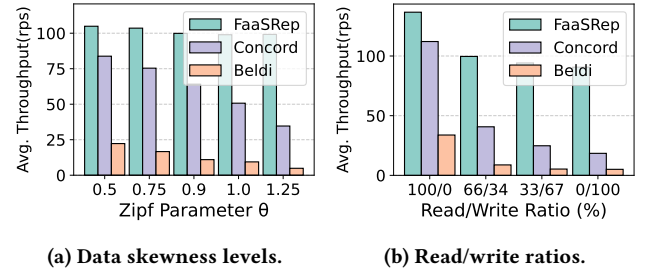


Figure 14: Performance comparison with varying read/write ratios or data skewness.

8.3 Varying Data Access Types

In this section, we evaluate the performance of FaaSRep and baselines under two critical dimensions of data access: data skewness (varying the Zipf factor θ from 0.5 to 1.25) and the ratio of read operations to write operations (ranging from read-only to write-only). From this section to Section 8.5, we use c4 with 32 clients as the workload for evaluation, serving as a moderate-intensity workload.

Impact of Data Skewness. As shown in Figure 14a, both Concord and Beldi experience a noticeable decline in throughput as the Zipf factor rises. Because higher skew increases contention on hot data items and leads to more frequent aborts. Beldi performs the worst overall due to the high latency of its purely remote locking mechanism, while Concord shows a comparable performance to FaaSRep with a low Zipf factor, as its cache coherence protocol effectively reduces remote data access.

For all skewness levels, FaaSRep consistently outperforms the baselines and exhibits more stable performance, declining only slightly from 104.9 to 99.14 rps. This resilience stems from its repair mechanism, whose overhead of building the dependency graph and fetching dependencies during repair are primarily determined by the number of data accesses, not the access pattern.

Impact of Read/Write Ratio. Figure 14b presents results under varying read/write ratios. As conflicts come from writes, increasing write proportion leads to a similar trend of performance decline to

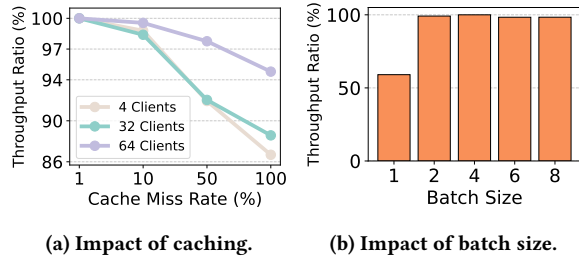


Figure 15: Effects of different caching and batching settings.

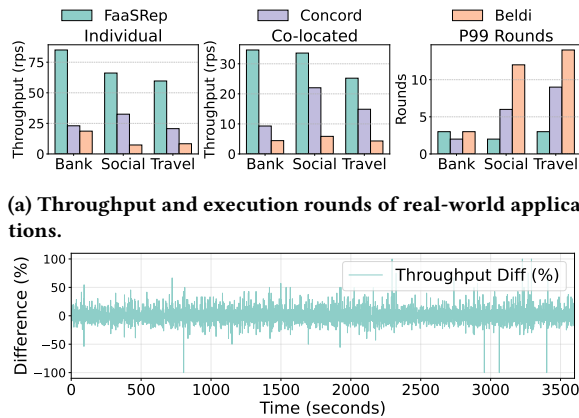
(b) Throughput Difference of weakened FaaSRep and trace input using *Travel Reservation*.

Figure 16: Evaluation with real-world applications and trace.

Figure 14a. Notably, a significant drop occurs when shifting from a pure read-only workload to a mixed workload. That's because read-only scenes are contention-free, allowing transactions to execute in parallel. In read-only scenarios, FaaSRep still performs best, as its batch commit strategy amortizes communication overhead in contrast to the per-transaction strategy employed by baselines.

8.4 Sensitivity Analysis

FaaSRep is derived from OCC, it also leverages caching and batching to enhance performance. In this section, we change configurations of these two methods to evaluate their impact.

First, we control the cache miss rate for each data access, and calculate the throughput degradation compared to the case with 1% miss rate. Figure 15a shows the results. Throughput drops as the miss rate increases, but less than 13.3% in all settings. It indicates that FaaSRep's performance improvement is not mainly from caching, but from its effective repair mechanism.

For batching, we change the batch size from 1 to 8, and measure the throughput ratio compared to the best case. Figure 15b shows that the main improvement occurs when the batch size increases to 2, with an increase of 67.9%. The subsequent growth shows a diminishing marginal effect, as larger batch is harder to fill, leading to untimely validation.

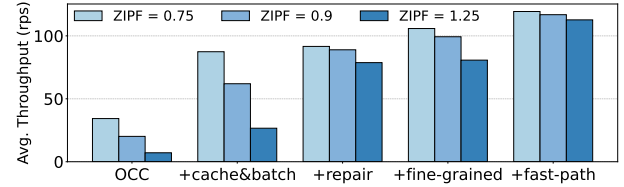


Figure 17: Design breakdown of FaaSRep.

8.5 Ablation Study

Figure 17 illustrates the impact of FaaSRep's key design components under varying contention levels. When no additional mechanisms are applied, FaaSRep degrades to OCC. In this setup, enabling caching and batching can improve the performance by 154.8-274.3%. The repair mechanism yields further throughput gains ranging from 4.83% to 195.63%. This benefit becomes especially pronounced as contention intensifies, which can be properly addressed by FaaSRep.

Moreover, enabling fine-grained repair (Section 6.1) contributes an additional 2.52-15.51% improvement. At this time, low-contention scenario benefits more, where a higher proportion of functions remain unaffected by conflicts and can be skipped during the repair phase, thereby further reducing unnecessary computation.

Finally, when the fast-path optimization is enabled, the throughput increases by another 12.7-39.6%, showing the effectiveness of bypassing the control plane. The benefit is also more noticeable under high contention, introducing heavy load to the control plane.

8.6 Performance in Real-world Scenarios

In the following subsections, we use the real-world applications described in Section 8.1. For each application, we allocate 32 clients to generate requests except for the trace experiment later. Besides, some requests may trigger aborts due to insufficient resources, such as overbooking flights in the travel reservation application.

Figure 16a shows the performance of systems with these applications. FaaSRep consistently outperforms the two baselines by a large margin, achieving 2.03-3.69 $\times$ (4.55-9.01 $\times$) the throughput of Concord (Beldi) when the applications are individually deployed. In the co-located scenario, this improvement comes to 1.52-3.72 $\times$ (5.83-7.81 $\times$) compared to Concord (Beldi). This improvement comes from two aspects: one is the bounded retry times achieved by the repair mechanism, the other is the alleviated control plane pressure by the fast-path. The latter part plays a more important role in the co-located scenario, as the control plane is handling requests from multiple applications concurrently.

The 99th-percentile rounds to completion offer deeper insight into how the systems handle high contention. In the travel reservation workload, the worst-case executions for Concord and Beldi require 9 and 14 rounds, respectively. In contrast, FaaSRep completes even the most contended requests in just 3 rounds, strictly bounded by the design of the pessimistic mode.

Finally, we evaluate FaaSRep under a 1-hour high-load trace from Azure [71, 75] to verify its practicality. Since proactive aborts depend heavily on specific application logic and data access patterns, we force every request to be fallbacked to pessimistic mode, creating a "weakened" version of FaaSRep to rigorously test the overhead. Figure 16b shows the result. For 79.4% of the duration,

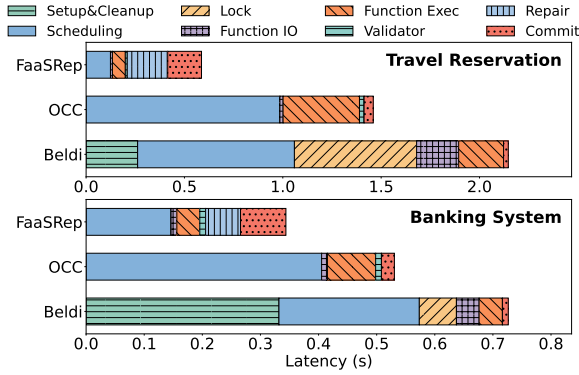


Figure 18: Execution time breakdown for transaction request.

the system's processing rate remains within 15.0% of the incoming trace rate. Significant lag (exceeding 20.0%) occurs in only 3.1% of the time, demonstrating that FaaSRep maintains robustness and can adapt to real-world workloads.

8.7 Execution Time Breakdown

In this subsection, we break down the average execution time of requests to analyze the factors contributing to performance differences among systems. Travel reservation represents a high-conflict scenario, while banking system brings fewer conflicts.

As shown in the top half of Figure 18, although OCC leverages cache to achieve a lower function IO latency than Beldi, both of them suffer from high scheduling overhead and function execution time caused by the contention-based retries. Beldi faces an additional penalty from its remote lock mechanism, surpassing the actual function execution time. Furthermore, a notable portion of Beldi's latency is consumed by the cleanup of lock entries. In contrast, while FaaSRep introduces specific overheads for repair and sequential commit, these costs are significantly outweighed by the savings achieved from eliminating full re-runs.

In the low-contention scenario, OCC and Beldi's performance improves significantly as the probability of "wasted work" drops. For FaaSRep, while its overhead of consistency protocol becomes more visible as contention vanishes, it still maintains the lowest latency. However, in extremely simplified environments like microbenchmarks (e.g., workload c2 in Section 8.2), FaaSRep's performance advantage may become less pronounced.

A potential source of overhead in FaaSRep is the construction of dependency graph, which is well-contained for several reasons. First, by delaying the construction until the validation phase, FaaSRep avoids most of the runtime communication overhead. Second, the batch size for each construction is capped and, as shown in Section 8.5, is often smaller than the maximum. Consequently, the time spent on dependency construction (contained in the "Validator" component) accounts for only a small fraction of total latency.

8.8 FaaSRep Overhead

In this experiment, we re-ran three real-world applications on the cluster and collected the resource usage difference between FaaSRep and OCC, to assess the overhead introduced by FaaSRep. The

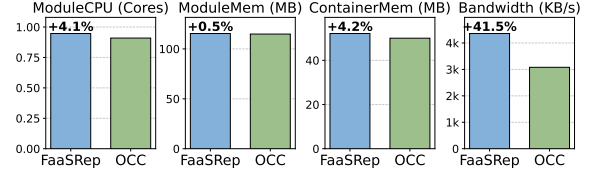


Figure 19: Display of FaaSRep overhead.

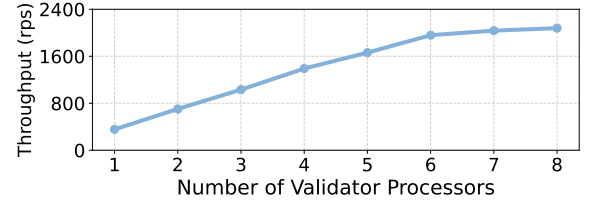


Figure 20: The processing capability of FaaSRep validator with different numbers of workers.

results are depicted in Figure 19. For system modules, FaaSRep only introduces an additional 4.1% CPU and 0.5% memory usage. For function containers, FaaSRep's repair sidecar increases the memory usage by 4.2%. Although the bandwidth usage increases by 41.5%, there's only a 1278.83 KB/s increase in absolute value. Compared with the significant performance improvement brought by FaaSRep, the overhead is acceptable.

8.9 Scalability of FaaSRep

FaaSRep's function execution framework adopts a similar architecture as prior systems [45, 71, 72]. The only new potential scalability bottleneck lies in the validator, which introduces a sequential stage for transaction validation and commit. To address this concern, we evaluate the validator's scalability under increasing load.

Figure 20 shows the throughput of the FaaSRep validator with varying processors. The validator is saturated when processors are 8, with a peak throughput of 2078.47 rps, which is already much higher than most real-world serverless traces such as the Azure Function trace [57], with no more than 500 rps as the peak throughput. What's more, many commercial serverless platforms also limit the maximum concurrency, such as AWS Lambda's default limit of 1000 concurrent functions per account [3].

9 Conclusion

In this paper, we present FaaSRep, a high-performance transactional system for serverless workflows with strict serializability. It leverages transaction repair to offer an effective, scheduled retry mechanism when conflicts occur. Specifically, it delays dependency construction at validation time to avoid runtime overhead, and proposes fine-grained function-level repair with a peer-to-peer fast-path to minimize control plane involvement. Moreover, it introduces a hybrid repair model to correctly handle proactive aborts of applications. Our evaluation demonstrates that FaaSRep significantly improves system throughput and reduces the response latency compared to existing solutions with negligible overhead.

References

- [1] 2025. AWS example for serverless transactional workflows. <https://aws.amazon.com/cn/blogs/compute/building-a-serverless-distributed-application-using-a-saga-orchestration-pattern/>.
- [2] 2025. AWS Lambda: execute code without thinking about servers. <https://aws.amazon.com/cn/lambda/>.
- [3] 2025. AWS Lambda quotas: limits related to AWS Lambda. <https://docs.amazonaws.cn/lambda/latest/dg/gettingstarted-limits.html>.
- [4] 2025. Azure Functions: Execute event-driven serverless code with an end-to-end development experience. <https://azure.microsoft.com/en-us/products/function/s/>.
- [5] 2025. DeathStarBench: A Benchmark Suite for Microservices. <https://github.com/delimitrou/DeathStarBench/>.
- [6] 2025. Docker: Accelerated Container Application Development. <https://www.docker.com/>.
- [7] 2025. Gevent: A coroutine-based Python networking library. <https://www.gevent.org/>.
- [8] 2025. Microservices-Based Banking Systems: Enabling Innovation Through Architectural Independence. <https://bicon.li/2025/05/01/microservices-based-banking-systems/>.
- [9] 2025. Redis: In-memory data structure store, used as a database, cache, and message broker. <https://redis.io/>.
- [10] 2025. ScyllaDB | Monstrously Fast + Scalable NoSQL. <https://www.scylladb.com/>.
- [11] 2025. Serverless App Engine of Aliyun. <https://www.aliyun.com/product/alibabacloud/sae>.
- [12] 2025. Tutorial for detecting and blurring offensive images using Cloud Functions. <https://cloud.google.com/functions/docs/tutorials/imagemagick>.
- [13] 2025. tySupermarket for worry-free purchasing. <https://ztf58.cn/>.
- [14] 2025. Use FFmpeg in Function Compute to process audio and video files in Function Compute. <https://www.alibabacloud.com/help/zh/functioncompute/fc-2-0/use-cases/use-ffmpeg-in-function-compute-to-process-audio-and-video-files>.
- [15] Mainak Adhikari, Tarachand Amgoth, and Satish Narayana Srirama. 2019. A survey on scheduling strategies for workflows in cloud environment and emerging trends. *ACM Computing Surveys (CSUR)* 52, 4 (2019), 1–36.
- [16] Marcos K Aguilera, Arif Merchant, Mehul Shah, Alistair Veitch, and Christos Karamanolis. 2007. Sinfonia: a new paradigm for building scalable distributed systems. *ACM SIGOPS Operating Systems Review* 41, 6 (2007), 159–174.
- [17] Hesam Nejati Sharif Aldin, Hossein Deldari, Mohammad Hossein Moattar, and Mostafa Razavi Ghods. 2020. Strict timed causal consistency as a hybrid consistency model in the cloud environment. *Future Generation Computer Systems* 105 (2020), 259–274.
- [18] Ahsan Ali, Riccardo Pincirol, Feng Yan, and Evgenia Smirni. 2020. Batch: Machine learning inference serving on serverless platforms with adaptive batching. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [19] Sadjad Fouladi and. 2019. From Laptop to Lambda: Outsourcing Everyday Jobs to Thousands of Transient. In *Proceedings of the 2019 USENIX Annual Technical Conference, USENIX, Dahlia Malkhi and (Ed.)*. USENIX Association, 475–488.
- [20] Lixiang Ao, Liz Izhikevich, Geoffrey M. Voelker, and George Porter. 2018. Sprocket: A Serverless Video Processing Framework. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC 2018, Carlsbad, CA, USA, October 11-13, 2018*. ACM, 263–274.
- [21] Robert L Axtell. 2001. Zipf distribution of US firm sizes. *science* 293, 5536 (2001), 1818–1820.
- [22] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. 1995. A critique of ANSI SQL isolation levels. *ACM SIGMOD Record* 24, 2 (1995), 1–10.
- [23] Philip A. Bernstein, David W. Shipman, and Wing S. Wong. 1979. Formal aspects of serializability in database concurrency control. *IEEE Transactions on Software Engineering* 3 (1979), 203–216.
- [24] João Carreira, Pedro Fonseca, Alexey Tumanov, Andrew Zhang, and Randy H. Katz. 2019. Cirrus: a Serverless Framework for End-to-end ML Workflows. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC 2019, Santa Cruz, CA, USA, November 20-23, 2019*. ACM, 13–24. doi:10.1145/3357223.3362711
- [25] Benjamin Carver, Jingyuan Zhang, Ao Wang, Ali Anwar, Panrui Wu, and Yue Cheng. 2020. Wukong: a scalable and locality-enhanced framework for serverless parallel computing. In *SoCC ’20: ACM Symposium on Cloud Computing, Virtual Event, USA, October 19-21, 2020*, Rodrigo Fonseca, Christina Delimitrou, and Beng Chin Ooi (Eds.). ACM, 1–15. doi:10.1145/3419111.3421286
- [26] Youmin Chen, Xiangyao Yu, Paraschos Koutris, Andrea C Arpaci-Dusseau, Remzi H Arpaci-Dusseau, and Jiwu Shu. 2022. Plor: General transactions with predictable, low tail latency. In *Proceedings of the 2022 International Conference on Management of Data*. 19–33.
- [27] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.
- [28] Marcin Copik, Konstantin Taranov, Alexandru Calotoiu, and Torsten Hoefler. 2023. rFaaS: Enabling high performance serverless with RDMA and leases. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 897–907.
- [29] Mohammad Dashti, Sachin Basil John, Amir Shaikhha, and Christoph Koch. 2017. Transaction repair for multi-version concurrency control. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 235–250.
- [30] Shlomi Dolev, Amos Israeli, and Shlomo Moran. 1993. Self-stabilization of dynamic systems assuming only read/write atomicity. *Distributed Computing* 7, 1 (1993), 3–16.
- [31] Zhiyuan Dong, Zhaoguo Wang, Xiaodong Zhang, Xian Xu, Changgeng Zhao, Haibo Chen, Aurojit Panda, and Jinyang Li. 2023. Fine-grained re-execution for efficient batched commit of distributed transactions. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1930–1943.
- [32] Aleksandar Dragojević, Dushyanth Narayanan, Edmund B Nightingale, Matthew Renzelmann, Alex Shamis, Anirudh Badam, and Miguel Castro. 2015. No compromises: distributed transactions with consistency, availability, and performance. In *Proceedings of the 25th symposium on operating systems principles*. 54–70.
- [33] Alexander Fuerst and Prateek Sharma. 2022. Locality-aware load-balancing for serverless clusters. In *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*. 227–239.
- [34] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyali Rath, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.
- [35] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. 2016. RDMA over commodity ethernet at scale. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 202–215.
- [36] Zhihan Guo, Kan Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing locks as early as you can: Reducing contention of hotspots by violating two-phase locking. In *Proceedings of the 2021 International Conference on Management of Data*. 658–670.
- [37] Zhipeng Jia and Emmett Witchel. 2021. Boki: Stateful serverless computing with shared logs. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 691–707.
- [38] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. 2019. Cloud programming simplified: A berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383* (2019).
- [39] Kostis Kaffes, Neeraja J Yadwadkar, and Christos Kozyrakis. 2022. Hermod: principled and practical scheduling for serverless functions. In *Proceedings of the 13th Symposium on Cloud Computing*. 289–305.
- [40] Anuj Kalia, Michael Kaminsky, and David G Andersen. 2016. {FaSST}: Fast, scalable and simple distributed transactions with {Two-Sided}({{{{RDMA}}}}) datagram {RPCs}. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. 185–201.
- [41] Gurkirat Kaur and Manju Bala. 2013. Rdma over converged ethernet: A review. *International Journal of Advances in Engineering & Technology* 6, 4 (2013), 1890.
- [42] Hsiang-Tsung Kung and John T Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems (TODS)* 6, 2 (1981), 213–226.
- [43] Ziliang Lai, Hua Fan, Wenchao Zhou, Zhanfeng Ma, Xiang Peng, Feifei Li, and Eric Lo. 2023. Knock out 2pc with practicality intact: a high-performance and general distributed transaction protocol. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2317–2331.
- [44] Zijun Li, Linsong Guo, Jiagan Cheng, Quan Chen, Bingsheng He, and Minyi Guo. 2022. The serverless computing survey: A technical primer for design architecture. *ACM Computing Surveys (CSUR)* 54, 10s (2022), 1–34.
- [45] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. 2022. Faasflow: Enable efficient workflow execution for function-as-a-service. In *Proceedings of the 27th acm international conference on architectural support for programming languages and operating systems*. 782–796.
- [46] Zijun Li, Chuhao Xu, Quan Chen, Jieru Zhao, Chen Chen, and Minyi Guo. 2023. Dataflow: Exploiting the data-flow paradigm for serverless workflow orchestration. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. 57–72.
- [47] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *European conference on computer vision*. Springer, 740–755.
- [48] Guowei Liu, Laiping Zhao, Yiming Li, Zhaolin Duan, Sheng Chen, Yitao Hu, Zhiyuan Su, and Wenyu Qu. 2024. Fuyao: Dpu-enabled direct data transfer for serverless computing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 431–447.
- [49] Qichang Liu, Yue Cheng, Haiying Shen, Ao Wang, and Bharathan Balaji. 2025. Concurrency-Informed Orchestration for Serverless Functions. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming*

- Languages and Operating Systems, Volume 2*. 147–161.
- [50] Yushi Liu, Shixuan Sun, Zijun Li, Quan Chen, Sen Gao, Bingsheng He, Chao Li, and Minyi Guo. 2024. FaaSGraph: Enabling scalable, efficient, and cost-effective graph processing with serverless computing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 385–400.
- [51] Fangming Lu, Xingda Wei, Zhuobin Huang, Rong Chen, Minyu Wu, and Haibo Chen. 2024. Serialization/deserialization-free state transfer in serverless workflows. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 132–147.
- [52] Taras Lykhenko, Rafael Soares, and Luis Rodrigues. 2021. Faastcc: Efficient transactional causal consistency for serverless computing. In *Proceedings of the 22nd International Middleware Conference*. 159–171.
- [53] Ashraf Mahgoub, Karthick Shankar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. 2021. {SONIC}: Application-aware data passing for chained serverless applications. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 285–301.
- [54] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. 2022. {ORION} and the three rights: Sizing, bundling, and prewarming for serverless {DAGs}. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 303–320.
- [55] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. 2012. Cache craftiness for fast multicore key-value storage. In *Proceedings of the 7th ACM european conference on Computer Systems*. 183–196.
- [56] Sheng Qi, Xuanzhe Liu, and Xin Jin. 2023. Halfmoon: Log-optimal fault-tolerant stateful serverless computing. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 314–330.
- [57] Mohammad Shahradd, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)*. 205–218.
- [58] Abraham Silberschatz, James L. Peterson, and Peter B. Galvin. 1991. *Operating system concepts*. Addison-Wesley Longman Publishing Co., Inc.
- [59] Swaminathan Sivasubramanian. 2012. Amazon dynamoDB: a seamlessly scalable non-relational database service. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 729–730.
- [60] Rafael Soares and Luis Rodrigues. 2023. Multi-Consistency Transactional Support for Function-as-a-Service. In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*. IEEE, 194–196.
- [61] Vikram Sreekanti, Chenggang Wu, Saurav Chhatrapati, Joseph E. Gonzalez, Joseph M. Hellerstein, and Jose M. Faleiro. 2020. A fault-tolerance shim for serverless computing. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–15.
- [62] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Jose M. Faleiro, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. 2020. Cloudburst: Stateful functions-as-a-service. *arXiv preprint arXiv:2001.04592* (2020).
- [63] Jovan Stojkovic, Chloe Alverti, Alan Andrade, Nikoleta Iliakopoulou, Hubertus Franke, Tianyin Xu, and Josep Torrellas. 2025. Concord: Rethinking Distributed Coherence for Software Caches in Serverless Environments. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 458–473.
- [64] Dixon Tang and Aaron J. Elmore. 2018. Toward coordination-free and reconfigurable mixed concurrency control. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 809–822.
- [65] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD international conference on management of data*. 1–12.
- [66] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon aurora: Design considerations for high throughput cloud-native relational databases. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1041–1052.
- [67] Tianzheng Wang and Hideaki Kimura. 2016. Mostly-optimistic concurrency control for highly contended dynamic workloads on a thousand cores. *Proceedings of the VLDB Endowment* 10, 2 (2016), 49–60.
- [68] Xingwei Wang, Hong Zhao, and Jiakeng Zhu. 1993. GRPC: A communication cooperation mechanism in distributed systems. *ACM SIGOPS Operating Systems Review* 27, 3 (1993), 75–86.
- [69] Chenggang Wu, Vikram Sreekanti, and Joseph M. Hellerstein. 2020. Transactional causal consistency for serverless computing. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 83–97.
- [70] Chao Xie, Chunzhi Su, Cody Little, Lorenzo Alvisi, Manos Kapritsos, and Yang Wang. 2015. High-performance ACID via modular concurrency control. In *Proceedings of the 25th Symposium on Operating Systems Principles*. 279–294.
- [71] Chuhao Xu, Yiyu Liu, Zijun Li, Quan Chen, Han Zhao, Deze Zeng, Qian Peng, Xueqi Wu, Haifeng Zhao, Senbo Fu, et al. 2024. Faasmem: Improving memory efficiency of serverless computing with memory pool architecture. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 331–348.
- [72] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. 2020. Fault-tolerant and transactional stateful serverless workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 1187–1204.
- [73] Haoran Zhang, Shuai Mu, Sebastian Angel, and Vincent Liu. 2025. Causalmesh: A causal cache for stateful serverless computing. *ACM SIGMOD Record* 54, 1 (2025), 48–57.
- [74] Qian Zhang, Jingyao Li, Hongyao Zhao, Quanqing Xu, Wei Lu, Jinliang Xiao, Fusheng Han, Chuanhui Yang, and Xiaoyong Du. 2023. Efficient distributed transaction processing in heterogeneous networks. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1372–1385.
- [75] Yanqi Zhang, Inigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and cheaper serverless computing on harvested resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 724–739.
- [76] Jianjun Zhao, Haikun Liu, Shuhao Zhang, Haodi Lu, Yancan Mao, Zhuohui Duan, Xiaofei Liao, and Hai Jin. 2025. Towards {High-Performance} Transactional Stateful Serverless Workflows with {Affinity-Aware} Leasing. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 1535–1551.